

TD de Programmation Orientée Objet

Licence d'Informatique*

Semaine du 30 mars 2026

Utilisation des Stream

L'utilisation de l'interface `Stream` de la bibliothèque de classe JAVA repose sur les mêmes principes que celle de la classe `Factory` vue en cours. L'interface `Stream` reprend l'ensemble des fonctionnalités de la classe `Factory` et en ajoute un grand nombre de nouvelles permettant ainsi d'effectuer des calculs de manière différente de ce que nous avons pu voir jusqu'à présent. En particulier, l'interface `Stream` possède des méthodes :

- `public Iterator<E> iterator()`
- `public Stream<E> filter(Predicate<? super E> p)`
- `public <R> Stream<R> map(Function<? super T, ? extends R> mapper)`

dont le fonctionnement est identique à celui des méthodes de même nom de la classe `Factory`.

L'interface `Stream` permet de plus d'effectuer certains calculs en parallèle lorsque la source de données s'y prête, cependant nous n'aborderons pas cet aspect dans le cadre de ce cours.

Opérations intermédiaires et opérations terminales

Pour les stream on distingue les opérations intermédiaires (i.e. *intermediate operations*) qui produisent un nouveau `Stream` des opérations terminales (i.e. *terminal operations*) qui produisent un résultat final.

Jusqu'à présent nous avons vu deux opérations intermédiaires : `filter` et `map` et une opération terminale : `iterator`.

Les opérations intermédiaires n'effectuent aucun calcul, elles se contentent de produire de nouveaux `Stream` capables d'effectuer un calcul modifié en fonction des caractéristiques de l'opération intermédiaire. Par contre, l'exécution d'une opération terminale va déclencher le calcul et produire un résultat (autre qu'un `Stream` : un `Iterator` pour la méthode `iterator`, un entier pour la méthode `count`, un élément du stream pour la méthode `max`). Ce qualificatif *terminal* indique aussi qu'après avoir exécuter cette opération, le `Stream` auquel s'applique l'opération terminale (ainsi que tous les `Stream` intermédiaires) qui avait été construit ne pourra plus être utilisé : pour effectuer un nouveau calcul sur les mêmes données un nouveau `Stream` devra être construit.

À titre d'exemple, vous pouvez compiler et exécuter le code suivant :

```
Stream<Direction> s1 = Stream.of(Direction.values());
Stream<Piece> s2 = s1.map(d -> p.pieceSuivante(d));
Stream<Piece> s3 = s2.filter(voisine -> voisine != null);
System.out.println("Execution_de_l'operation_terminale_iterator()");
Iterator<Piece> iterP = s3.iterator();
System.out.println("Tentative_de_reutiliser_un_Stream_intermediaire");
s2.filter(voisine -> voisine != null);
// Exception in thread "main" java.lang.IllegalStateException:
// stream has already been operated upon or closed
while (iterP.hasNext()) {
    System.out.println(":" + iterP.next());
}
```

Parmi les autres opérations terminales utiles, on retiendra :

`toList()` qui renvoie une liste contenant tous les éléments du stream

`toArray()` qui renvoie un tableau contenant tous les éléments du stream

*Marc Champesme – Département d'Informatique – Institut Galilée

boolean anyMatch(Predicate<? super T> predicate) qui renvoie `true` si au moins un élément du stream satisfait le prédicat donné en argument

boolean allMatch(Predicate<? super T> predicate) qui renvoie `true` si tous les éléments du stream satisfont le prédicat donné en argument

Exercice 1 Nouvelle implémentation de la méthode `choisirPiece` de la classe `StrategieMaxPredicate` Dans le sujet de partiel la méthode `choisirPiece` est présentée de la manière suivante :

Pour choisir la pièce vers laquelle un personnage se déplace on utilise le design pattern Strategy et on définit une interface `StrategieDeplacement` possédant l'unique méthode :

Piece choisirPiece(Piece p, Predicate<? super ObjetZork> pred)

1. *Donnez l'implémentation complète d'une classe `StrategieMaxPredicate` implémentant l'interface `StrategieDeplacement` de manière à ce que sa méthode `choisirPiece` possède les propriétés suivantes :*

- *si la Piece donnée en paramètre ne possède pas de Piece voisine la méthode renvoie la valeur `null`;*
- *la Piece renvoyée est une pièce voisine de la Piece donnée en paramètre;*
- *parmi toutes les Piece voisines de la Piece donnée en paramètre, la Piece renvoyée est celle contenant le plus grand nombre d'ObjetZork satisfaisant le prédicat passé en paramètre.*

Pour cette classe, vous appliquerez le design pattern Singleton.

Une implémentation possible de la classe `StrategieMaxPredicate` est :

```
public class StrategieMaxPredicate implements StrategieDeplacement {
    public static final StrategieMaxPred INSTANCE = new StrategieMaxPred();

    private StrategieMaxPred() {
    }

    @Override
    public Piece choisirPiece(Piece p, Predicate<? super ObjetZork> pred) {
        Piece result = null;
        int max = 0;
        for (Direction d : Direction.values()) {
            int count = 0;
            Piece voisine = p.pieceSuivante(d);
            if (voisine != null) {
                for (ObjetZork oz : voisine) {
                    if (pred.test(oz)) {
                        count++;
                    }
                }
                if (count > max) {
                    max = count;
                    result = voisine;
                }
            }
        }
        return result;
    }
}
```

1. L'interface `Stream` possède une méthode `of` définie de la manière suivante¹ :

```
/**
 * Returns a sequential ordered stream whose elements are
 * the specified values.
 */
```

1. En réalité la définition de la méthode est `static <T> Stream<T> of(T... values)` ce qui correspond à une méthode à nombre variable d'arguments, cependant, en pratique, une telle méthode peut toujours être appelée avec un tableau en argument.

```

    * @param <T> the type of stream elements
    * @param values the elements of the new stream
    * @return the new stream
    */
    static <T> Stream<T> of(T[] values) { ... }

```

Donnez le code permettant de créer un Stream dont les valeurs sont l'ensemble des instances de la classe enum Direction.

2. En déduire le code permettant de créer un Stream contenant les Piece voisines d'une pièce p (indication : utilisez les méthodes map et filter de l'interface Stream).
3. En supposant que la classe Piece possède un attribut private List<ObjetZork> lObj donner l'implémentation d'une (nouvelle) méthode Stream<ObjetZork> stream() de la classe Piece, qui renvoie un Stream de l'ensemble des ObjetZork présents dans la pièce (on rappelle que l'interface Collection possède une méthode Stream<E> stream()).
4. L'interface Stream possède une méthode count définie de la manière suivante :

```

/**
 * Returns the count of elements in this stream.
 *
 * This is a terminal operation.
 *
 * @return the count of elements in this stream
 */
long count();

```

En utilisant la méthode stream de la classe Piece et la méthode count de l'interface Stream, définissez une méthode long countPred(Piece p, Predicate<? super ObjetZork> pred) permettant de calculer le nombre d'ObjetZork présents dans une pièce p et satisfaisant le prédicat pred.

5. L'interface Stream possède une méthode max définie de la manière suivante :

```

/**
 * Returns the maximum element of this stream according to the provided
 * Comparator.
 *
 * This is a terminal operation.
 *
 * @param comparator a Comparator to compare elements of this stream
 * @return an Optional describing the maximum element of this stream,
 *         or an empty Optional if the stream is empty
 * @throws NullPointerException if the maximum element is null
 */
Optional<T> max(Comparator<? super T> comparator);

```

La classe Optional est une classe qui permet de traiter les cas où une méthode ne peut pas de donner de réponse correcte en évitant de renvoyer une valeur null dans ce cas (pour max ce serait le cas si le Stream est vide). Pour cela une instance de Optional encapsule une instance et possède deux méthodes pour tester si la valeur encapsulée est pertinente (isPresent()) ou non (isEmpty()). Si (et seulement si) la valeur est pertinente on y accède avec la méthode T get().

On rappelle que l'interface Comparator est une interface fonctionnelle dont l'unique méthode est int compare(T o1, T o2) qui renvoie un nombre < 0 si o1 < o2, égal à 0 si o1 = o2 et un nombre > 0 si o1 > o2.

En déduire une nouvelle implémentation de la méthode choisirPiece utilisant les stream.

Exercice 2 L'opération intermédiaire flatMap.

Supposons que nous ayons une classe Etudiant, une classe GroupeTD possédant une méthode List<Etudiant> getEtudiants() qui renvoie une liste de tous les étudiants d'un groupe de TD et une classe Promo ayant une méthode List<GroupeTD> getAllGroups() qui renvoie une liste de tous les groupes de TD. Comment obtenir une liste de tous les étudiants d'une promo ?

Le code suivant utilisant map ne convient pas car on obtient en résultat une liste **de listes** d'étudiants :

```
Promo maPromo = ...
List<List<Etudiant>> listeEtu = maPromo
    .getAllGroups()
    .stream() // un stream de GroupeTD
    .map(gr -> gr.getEtudiants()) // un stream de
                                // listes d'etudiants
    .toList();
```

La méthode `flatMap` est définie de la manière suivante :

```
/**
 * Returns a stream consisting of the results of replacing each element of
 * this stream with the contents of a mapped stream produced by applying
 * the provided mapping function to each element.
 *
 * @param <R> The element type of the new stream
 * @param mapper a non-interfering, stateless function
 *               to apply to each element which produces
 *               a stream of new values
 * @return the new stream
 */
<R> Stream<R> flatMap(Function<? super T, ? extends Stream<? extends R>>
mapper);
```

Pour simplifier (en retirant les ?), l'entête de la méthode est :

```
<Stream<R> flatMap(Function<T, Stream<R>>> mapper);
```

Cette méthode prend donc en paramètre une fonction qui retourne pour chaque élément de type `T` du stream source, un stream d'éléments de type `R`, et concatène l'ensemble des streams obtenus en appliquant la fonction `mapper`. On peut donc l'appliquer dans l'exemple précédent :

```
Promo maPromo = ...
List<Etudiant> listeEtu = maPromo
    .getAllGroups()
    .stream()
    .flatMap(gr -> gr.getEtudiants().stream())
    .toList();
```

À l'aide de la méthode `flatMap` donnez le code permettant d'obtenir une liste de tous les `ObjetZork` contenus dans toutes les pièces voisines d'une pièce donnée.

Exercice 3 Utilisation de la méthode `collect`.

La méthode `collect` est une autre opération terminale importante de l'interface `Stream`, elle prend en paramètre une instance d'une classe implémentant l'interface `Collector` qui offre une grande flexibilité sur le choix de l'objet à produire. Pour simplifier l'usage de cette méthode la classe `Collectors` contient un grand nombre de méthodes `static`, qui renvoie des `Collector` couvrant les usages les plus courants² :

`toSet()` qui retourne un `Set` contenant l'ensemble des éléments du `Stream`.

```
Piece p = ...;
Set<ObjetZork> s = p.stream().collect(toSet());
```

`toList()` qui retourne une `List` contenant l'ensemble des éléments du `Stream`

```
Piece p = ...;
List<ObjetZork> s = p.stream().collect(toList());
```

2. Dans les exemples suivant, on supposera que l'on a ajouté la directive `import static java.util.stream.Collectors.*`

toCollection(Supplier<C> collectionFactory) qui retourne une `Collection` contenant l'ensemble des éléments du `Stream` en utilisant une fonction qui produit une nouvelle instance d'une classe implémentant l'interface `Collection` (l'intérêt est de pouvoir choisir la classe du résultat) :

```
Piece p = ...;
List<ObjetZork> s = p.stream()
    .collect(toCollection(() -> new LinkedList<>()));
```

toMap(Function<T, K> keyMapper, Function<T, U> valueMapper) qui retourne une `Map` dont les clés sont les valeurs renvoyées par la fonction `keyMapper` et leur associe la valeur renvoyée par le fonction `valueMapper`.

```
Piece p = ...;
Map<Direction, List<ObjetZork>> map = Stream.of(Direction.values())
    .filter(d -> p.pieceSuivante() != null)
    .collect(toMap(d -> p.pieceSuivante(d),
        d -> p.pieceSuivante(d)
            .stream()
            .toList()));
```

1. Quel est le résultat produit par le dernier exemple avec le `Collector toMap`?
2. Définissez une méthode `Map<Direction, List<ObjetZork>> getMapDirToOz(Piece p)` qui renvoie une `Map` associant à chaque direction la liste des `ObjetZork` présents dans la pièce voisine de la pièce spécifiée en prenant cette direction.
3. Définissez une méthode

```
Map<Direction, List<ObjetZork>>
    getMapPred(Piece p, Predicate<? super ObjetZork> pred)
```

qui renvoie une `Map` associant à chaque direction la liste des `ObjetZork` présents dans la pièce voisine de la pièce spécifiée en prenant cette direction et qui satisfont le prédicat spécifié.